

Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #11

Graph traversal

Agenda

- **Built-in predicates** (and relationship to graphs)
 - *findall* and more
- **Constructs and relationship with graphs**
 - For all is true
 - Application – same graph
- **Graph Traversal**
 - dfs
 - bfs (v1 – other versions later)

Built-in predicates

their utility in graph searching

- `findall` – selects all items (and only those items) with a certain property from a knowledge-base
- You must know the predicate to use it appropriately
 - Know HOW it is implemented
 - to utilize the strategy in graph problems

findall

- (example from Clocksin and Mellish)
- Suppose we have a knowledge base of some drinks lovers:
`% likes/2 (+Person, +Drink).`

```
likes(bill, wine).  
likes(dick, beer).  
likes(harry, beer).  
likes(john, beer).  
likes(peter, wine).  
likes(tom, beer).
```

```
[q] ?- findall(X, likes(X,beer), L).  
true, L=[dick,harry,john,tom].
```

findall – implementation

```
% findall/3 (+CollectedVar, +CollectorPred, -CollectorVar).
```

```
findall(X,G,_):-
    asserta(found(end)),
    G,
```

```
    asserta(found(X)),
    fail.
```

```
findall(_,_,L):-
    collect_found([],L).
```

```
collect_found(P, L):-
```

```
    retract(found(X)),
    X\=end,!,
```

```
    collect_found([X|P],L).
```

```
collect_found(L,L).
```

% uses an auxiliary knowledge base (**akb**) - found

% marker for the bottom of the **akb**

% = call(G), G is a predicate;

% its exe instantiates some argument X.

% puts G's instantiated argument on top of **akb**

% request for backtrack to G.

% starts collecting from the **akb**,

% initializes the partial result to empty list.

% extracts from top of the **akb**

% succeeds if a genuine data; fails when at the end

% adds it in front and go recurse

findall – implementation

```
% findall/3 (+CollectedVar, +CollectorPred, -CollectorVar).
```

```
findall(X,G,_):-  
    asserta(found(end)),  
    G,  
    asserta(found(X)),  
    fail.
```

```
findall(_,_ ,L):-  
    collect_found([],L).
```

```
collect_found(P,L):-  
    retract(found(X)),  
    X\=end,!,  
    collect_found([X|P],L).
```

```
collect_found(L,L).
```

findall – execution

```
findall(X,G,_):-
    asserta(found(end)),
    G,
    asserta(found(X)),
    fail.
findall(_,_,L):-
    collect_found([],L).

collect_found(P,L):-
    retract(found(X)),
    X\=end,!,
    collect_found([X|P],L).
collect_found(L,L).
```

```
[q] ?- findall(X, likes(X,beer), L).
```

% initial kb

```
likes(bill, wine).
likes(dick, beer).
likes(harry, beer).
likes(john, beer).
likes(peter, wine).
likes(tom, beer).
```

% akb

```
found(tom).
found(john).
found(harry).
found(dick).
found(end).
```

%created bottom-up↑ collected top-down↓

findall – related

- General call: `?- findall(X,G,L)`
 - where `X` is a variable
 - `G` a predicate and `X` occurs in `G`
 - `L` would collect all `X`'s so that `G`'s execution succeeds on `X`
- Creates the list `L` of `X`'s that satisfy `G`, in the order of occurrence in the original knowledge base
- Associate predicates:
- `setof(X,G,S)`
 - Creates the set of terms (standard order, without duplicates; represented also as list) of `X`'s that satisfy `G`. If none, fails.
 - **Differences to `findall`:**
 - Order: standard vs as found in the knowledge base
 - Duplicates: no vs found in the knowledge base
 - No term: fails vs empty list
- `bagof(X,G,S)`
 - Same as `setof` BUT list NOT ordered + may have duplicates
 - So same as `findall` but fails if no term matches

```
findall(X,G,L):- bagof(X,G,L),!.
```

```
findall(_,_,[]).
```


For all is true technique

- Construct to check if `for_all Predicate1`
`is_true Predicate2`
- Aims to identify/verify whether in all cases when P1 is true, P2 is true as well. Thus, checks $P1 \Rightarrow P2$.
- Template looks like:

```
for_all_is_true(X,Y) :-
    X,                                % calls X, a predicate,
                                     % whose execution may instantiate hidden args
    not(Y) , !,                       % calls Y in the same context. If succeeds, its negation fails,
                                     % and backtracks to X who is executed again, with a
                                     % new instance of args; generates a new context
                                     % the first context of X where Y fails, succeeds, and !
                                     % is irreversible crossed and the predicate fails
    fail.                             % overall.

                                     % if we reached here, in all contexts when X holds true
for_all_is_true(_,_) .% succeeds as well, therefore,  $X \Rightarrow Y$ 
```

For all is true in graphs context

- Given graphs G1 and G2 with neighbor and edge representations respectively, do they represent the same graph?

G1

neighbor(Node, List_of_Neighbors)

G2

edge(Node1, Node2)

- G1 and G2 same means $G1 \Leftrightarrow G2$ which is $(G1 \Rightarrow G2) + (G2 \Rightarrow G1)$

$(G1 \Rightarrow G2)$
for_all edges_in_G1
is_true edge_in_G2

$(G2 \Rightarrow G1)$
for_all edges_in_G2
is_true edge_in_G1

- Define the edges in the 2 representations:

An edge in G1: (defines the nondeterministic check for all edges)

```
is_edge_1(X, Y) :-
    neighbor(X, L),
    member(Y, L).
```

% to find all edges in G1, call nondeterminist(X,Y,free)
% finds first pair first (and next on backtrack), instantiates both X and L.
% nondeterm call on member; instantiate Y, one at a time, eventually all.

An edge in G2:

```
is_edge_2(X, Y) :-
    edge(X, Y);
    edge(Y, X).
```

Same graph?

- $(G1 \Rightarrow G2)$ $(G2 \Rightarrow G1)$
`for_all edges_in_G1` `for_all edges_in_G2`
`is_true edge_in_G2` `is_true edge_in_G1`
- Denote the `for_all_is_true` as `eq1` and `eq2` depending on the implication $(G1 \Rightarrow G2)$ respectively $(G2 \Rightarrow G1)$ we verify

`eq1:-`

```
is_edge_1(X,Y),
not(is_edge_2(X,Y)),!,
fail.
```

% for each edge in the first representation

% there is one edge in the other graph

% otherwise, we reach here, hence, fail.

`eq1.`

% if we get here, $G1 \Rightarrow G2$ shown

`eq2:-`

```
is_edge_2(X,Y),
not(is_edge_1(X,Y)),!,
fail.
```

`edge(a,b).`

`edge(b,a).`

`neighbor(a,[b]).`

`neighbor(b,[a]).`

`eq2.`

`same_graph:- eq1, eq2.`

`[q] ?- same_graph.`

Change representation of a graph

- Given graph **G1** a weighted, edge representation and we need to get **G2** with neighbor representation

G1

```
edge(a,b,15).
```

```
edge2neighb:-
```

```
    is_edge(X,_,_),
    not(neighbor(X,L)),
    forall(Z,convert(X,Z),L),
    assertz(neighbor(X,L)),
    fail.
```

```
edge2neighb.
```

```
convert(X,Z):-
```

```
    is_edge(X,Y,W),
    Z=p(Y,W).
```

- Note:**

```
forall(Z,succ(X,Z),L) same as forall(p(Y,W),is_edge(X,Y,W),L).
```

- Given **G2**, generate **G1**. Homework.

G2

```
neighbor(a,[p(b,15),...]).
```

Depth first search

- Assume undirected, unweighted graph, edge representation
- 2 stage process:
 1. 1st clause: make the trail placing in a queue the vertices in order of visitation (being in the additional knowledge base `vert` - hence a dynamic predicate - is as if we have a grey vertex).
 2. 2nd clause: when done, collect them.

```
dfs(X,_) :-
```

```
    assertz(vert(X)),           % place start/current node in Q
    is_edge(X,Y),               % take first/next neighbor of current node X
    not(vert(Y)),               % if already visited, backtrack to take another;
    dfs(Y,_) .                  % if not, continue from Y
```

```
dfs(_,L) :-
```

```
    assertz(vert(end)),
    collect([],L) .             % similar to collect in findall, but on vert akb.
```

- It is covering just the connected component of the starting vertex (is similar to `dfs_visit` in Cormen).
 - Need a wrapper to re-run it from all connected components (all remaining white nodes – color NOT implemented here).

Breadth first search

- Assume undirected, unweighted graph, edge representation
- Queue made by the akb named `node` (hence dynamic predicate)

```
bfs(X,_) :-
```

```
    assertz(node(X)),           % add at the end of the Q the current node
    node(Y),                   % reads from front of Q, gets Y instantiated (initially X=ONLY one in Q)
    is_edge(Y,Z),              % take first/next neighbor of current Y (gets Z instantiated);
                                % if none, backtrack and take next from Q
    not(node(Z)),               % if Z already visited, backtrack to take another;
    assertz(node(Z)),           % if not, put Z in Q and
    fail.                       % backtrack anyway to another neighbor of Y
```

```
bfs(_,L) :-
```

```
    assertz(node(end)),
    collect([],L).              % similar to collect in findall, but on node in akb.
```

- There is an issue with this implementation for some languages/dialects (due to an optimization strategy). Oral explanation. Fix it!.

BFS – queues explicit

- BFS with 3 arguments representing:
 - Candidate list (C_{and}) = queue = list of items we reached, yet not finalized to process = grey nodes. Implemented as incomplete lists. Why? Try to find out. Oral discussion!
 - Expanded list (E_{xp}) = list of items reached and processed = black nodes. Implemented as regular (complete) lists.
 - Result (R_{ez}) = nodes in order of bfs processing. Copy of Expanded list when processing ends.

```
% bfs /3 (+Queue, +ExpansionList, -Result).
```

```
[q] ?- bfs([a|_], [], R).
```

```
bfs (Cand, Exp, Exp) :-                % when Q is empty, end exe, the Exp becomes Rez
    var (Cand), !.
bfs ([X|Cand], Exp, R) :-              % else, take first from Q
    expand (X, Cand, Exp),              % and expand (put all white neighbors in Q) and
    bfs (Cand, [X|Exp], R).            % continue by moving it in expanded
                                       % = make it black
```

BFS – queues explicit - contd

- The expansion step (2-stage process) – decides which neighbors to add in Q
- dynamic predicate `desc`; potential neighbors stored in auxiliary knowledge base (akb)

```

expand(X,_,Exp):-
    is_edge(X,Z),                % nondeterministically take Z, first neighbor of X
                                % (eventually all of them)
    not(member(Z,Exp)),          % should NOT be already processed (not a black node)
    assertz(desc(Z)),            % potentially add it in Q
    fail.                        % backtrack to evaluate another neighbor of X

expand(_,Cand,_):-
    assertz(desc(end)),          % mark end of akb
    collect(Cand).

collect(Cand):-
    retract(desc(X)),            % take X and if not under processing (not a grey one)
    X\=end, !,                  % as long as akb not empty
    insert_IL(X,Cand),           % add in Q
    collect(Cand).              % continue. How is possible with the SAME argument?

collect(Cand).                  % end when akb empty

```


BFS – complete code

```
bfs (Cand, Exp, Exp) :-  
    var (Cand), !.  
bfs ([X|Cand], Exp, Rez) :-  
    expand (X, Cand, Exp),  
    bfs (Cand, [X|Exp], Rez).
```

```
expand (X, _, Exp) :-  
    is_edge (X, Z),  
    not (member (Z, Exp)),  
    assertz (desc (Z)),  
    fail.
```

```
expand (_, Cand, _) :-  
    assertz (desc (end)),  
    collect (Cand).
```

```
[q] ?- bfs ([a|_], [], Rez).
```

```
collect (Cand) :-  
    retract (desc (X)),  
    X \= end, !  
    insert_IL (X, Cand),  
    collect (Cand).  
collect (Cand).
```

```
insert_IL (X, [X|_]) :- !.  
insert_IL (X, [_|L]) :-  
    insert_IL (X, L).
```

BFS – execution

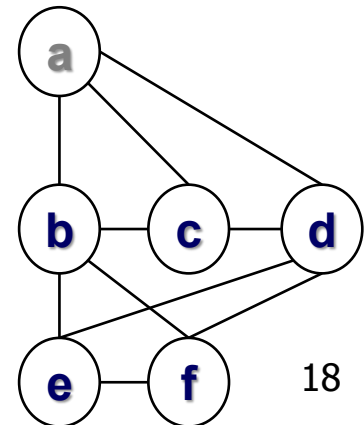
```
[q] ?- bfs([a|_], [], Rez) . % for the first graph – search for a path
```

Step1

Exp: []

Cand: [a|_]

```
edge(a,b,7) .  
edge(a,c,1) .  
edge(a,d,8) .  
edge(b,c,2) .  
edge(b,f,5) .  
edge(b,e,10) .  
edge(c,d,3) .  
edge(d,f,4) .  
edge(d,e,8) .  
edge(f,e,3) .
```



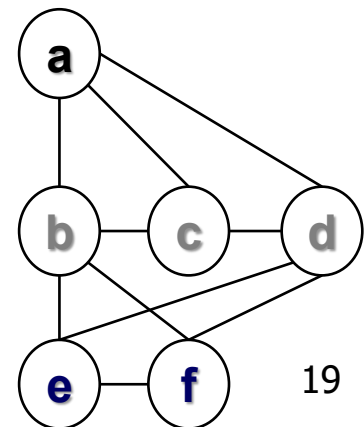
BFS – execution

[q] ?- bf_search([a|_], [], Rez) . % for the first graph – search for a path

	Step1	Step2
Exp:	[]	[a]
Cand:	[a _]	[b,c,d _]

```

edge(a,b,7) .
edge(a,c,1) .
edge(a,d,8) .
edge(b,c,2) .
edge(b,f,5) .
edge(b,e,10) .
edge(c,d,3) .
edge(d,f,4) .
edge(d,e,8) .
edge(f,e,3) .
    
```



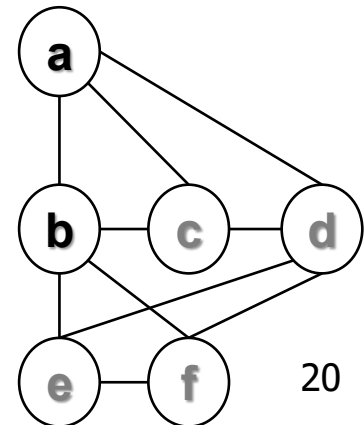
BFS – execution

[q] ?- bf_search([a|_], [], Rez) . % for the first graph – search for a path

	Step1	Step2	Step3
Exp:	[]	[a]	[b,a]
Cand:	[a _]	[b,c,d _]	[c,d,f,e _]

```

edge(a,b,7) .
edge(a,c,1) .
edge(a,d,8) .
edge(b,c,2) .
edge(b,f,5) .
edge(b,e,10) .
edge(c,d,3) .
edge(d,f,4) .
edge(d,e,8) .
edge(f,e,3) .
    
```



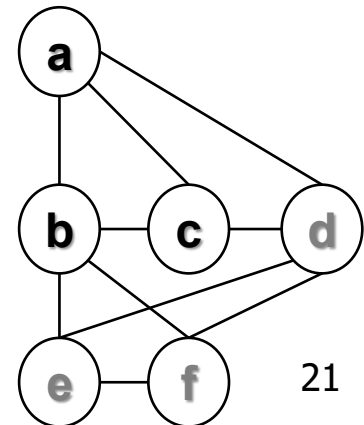
BFS – execution

[q] ?- bf_search([a|_], [], Rez) . % for the first graph – search for a path

	Step1	Step2	Step3	Step4
Exp:	[]	[a]	[b,a]	[c,b,a]
Cand:	[a _]	[b,c,d _]	[c,d,f,e _]	[d,f,e _]

```

edge(a,b,7) .
edge(a,c,1) .
edge(a,d,8) .
edge(b,c,2) .
edge(b,f,5) .
edge(b,e,10) .
edge(c,d,3) .
edge(d,f,4) .
edge(d,e,8) .
edge(f,e,3) .
    
```



BFS – execution

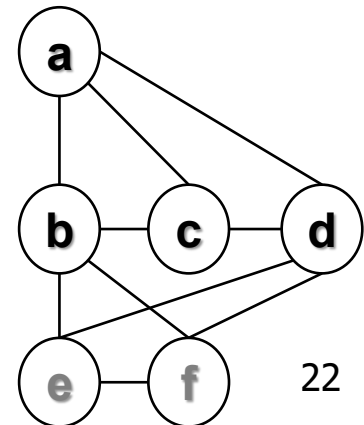
[q] ?- bf_search([a|_], [], Rez) . % for the first graph – search for a path

	Step1	Step2	Step3	Step4	Step5
Exp:	[]	[a]	[b,a]	[c,b,a]	[d,c,b,a]
Cand:	[a _]	[b,c,d _]	[c,d,f,e _]	[d,f,e _]	[f,e _]

```

edge(a,b,7) .
edge(a,c,1) .
edge(a,d,8) .
edge(b,c,2) .
edge(b,f,5) .
edge(b,e,10) .
edge(c,d,3) .
edge(d,f,4) .
edge(d,e,8) .
edge(f,e,3) .

```



BFS – execution

[q] ?- bf_search([a|_], [], Rez) . % for the first graph – search for a path

	Step1	Step2	Step3	Step4	Step5	Step6	Step7
Exp :	[]	[a]	[b,a]	[c,b,a]	[d,c,b,a]	[f,d,c,b,a]	[e,f,d,c,b,a] DONE!
Cand:	[a _]	[b,c,d _]	[c,d,f,e _]	[d,f,e _]	[f,e _]	[e _]	_

edge(a,b,7) .
 edge(a,c,1) .
 edge(a,d,8) .
 edge(b,c,2) .
 edge(b,f,5) .
 edge(b,e,10) .
 edge(c,d,3) .
 edge(d,f,4) .
 edge(d,e,8) .
 edge(f,e,3) .

